# ByteBreaker@DravidianLangTech 2026: XLM-RoBERTa Large with Sliding-Window Chunking and Top-$K$ Mean Pooling for Writing Style Classification

**Srinivasa Sai Chava**
Boston University
Massachusetts, USA
srinivassaichava@gmail.com

**Vinay Kumar Rangoori**
IIIT Dharwad
Karnataka, India
vinaykumarrangoori33@gmail.com

**Jigeesha Sai Surapaneni**
Dhanekula Institute of Engineering & Technology
Andhra Pradesh, India
jigeeshasurapaneni@gmail.com

**Shanmukha Sai Chava**
IIITDM Kancheepuram
Tamil Nadu, India
shanmukhasaichava@gmail.com

## Abstract

Identifying different writing styles in large chunks of text is difficult because writing styles vary in different sections of a document . Additionally , the writing styles associated with a text can be differentiated in only tiny and nuanced ways . In this paper, we describe ByteBreaker , the system we built for the Prompt Recovery for LLM Shared Task at *DravidianLangTech@ACL-2026* . The goal is to analyze the writing style in a specific document that a large language model (LLM) has written . The styles to choose from are categorized as : Authoritative,Formal, Humorous,Informal,Inspiring,Optimistic,Persuasive, Pessimistic, and Serious. Given that a number of documents exceed the 512 token limit of transformer models, we adopt a sliding-window method that breaks each document down into overlapping 512 token chunks , with a stride of 256 tokens . We fine-tune XLM-RoBERTa Large with just the rewritten "CHANGE STYLE" text , as that one has more distinct stylistic indicators . For prediction , we Top-K mean pool the chunk-level predictions, which puts more emphasis on the confident chunks as opposed to treating all chunks the same . To enhance consistency, we trained the model with five distinct random seeds and made three submission:a weighted ensemble(Run 1), a mean-guided single model (Run 2), and a Top-K-guided single model (Run 3). Among the three, Run 3 reached the highest macro F1 score of 0.3306, while Run 1 achieves the best accuraccy(0.3256) with a macro F1 of 0.3290.

## 1 Introduction

Writing style is also an important factor that determines how a message is received. Apart from the information conveyed in a piece of writing, factors such as tone, vocabulary, sentence structure, and formality level contribute to the way in which the message is received. The automatic classification of writing styles has thus become a significant area of research in natural language processing. With the increasing use of large language models (LLMs) for text generation and text rewriting, the analysis of stylistic intent has become even more important.

The Prompt Recovery task brings a novel and applicable variation to the style classification task. Rather than examining naturally written human text, the goal is to recover the style of writing used by the LLM to re-write a given document based on a given prompt. While the re-writing task can enhance the stylistic cues, it is still difficult to distinguish between styles that are very similar. There are some categories that have overlapping features, and the distinctions between them are not always obvious.

Another important challenge that comes into play is the fact that the documents are quite long. This means that the samples are also long, with several sections where stylistic expression can be varied. However, the fact that the models are based on the Transformer architecture means that there are input size restrictions. This makes it quite challenging to handle the entire document at once. At the same time, the stylistic signals are not always evenly distributed in the text.

In this paper, we propose ByteBreaker, a system that aims to overcome these difficulties using structured document processing and aggregation of model predictions. Instead of considering all parts of a document as equally important, our method is centered on finding and highlighting the most in-

formative parts of a document for decision-making. We also investigate various training methods to enhance consistency and stability.

By means of systematic experimentation, we examine the impact of design choices on performance in the Prompt Recovery environment. Our work underscores the significance of integrating training approaches with document-level prediction models and illustrates the significance of proper management of long-form text in writing style classification.

## 2 Related Work

Writing style analysis has been studied extensively in the context of authorship attribution and stylistic text classification. Early approaches relied on handcrafted linguistic features such as lexical patterns, syntactic structures, and functional word usage to capture stylistic differences between texts (Argamon et al., 2007; Koppel et al., 2009). Convolutional neural network-based methods were later introduced to model stylistic signals directly from text without manual feature engineering (Shrestha et al., 2017). While these approaches demonstrated promising results, they often struggled to capture deeper contextual relationships in long-form documents.

The introduction of transformer-based models significantly advanced text classification research. BERT (Devlin et al., 2019) and its improved variant RoBERTa (Liu et al., 2019) showed strong performance across various natural language understanding tasks by learning contextualized representations. XLM-RoBERTa (Conneau et al., 2020) further extended this capability to multilingual settings, making it suitable for diverse language tasks. The availability of robust transformer frameworks such as Hugging Face Transformers (Wolf et al., 2020) has enabled rapid development and fine-tuning of large-scale pre-trained models for downstream classification problems.

Despite their strong performance, standard transformer models are limited by a maximum input length of 512 tokens. Several approaches have been proposed to address long-document classification. Hierarchical attention networks model text at both sentence and document levels (Yang et al., 2016), while sliding-window fine-tuning strategies split documents into manageable segments and aggregate predictions at the document level (Sun et al., 2019). Alternatively, long-context transformer variants such as Longformer (Beltagy et al., 2020) and BigBird (Zaheer et al., 2020) modify the attention mechanism to efficiently process longer sequences.

In shared task settings and low-resource scenarios, ensemble strategies and multi-model training approaches have been widely adopted to improve robustness and generalization (Saumya et al., 2022; Fharook et al., 2022; Kavatagi et al., 2023). Research in code-mixed and multilingual text processing has further highlighted the effectiveness of transformer-based architectures in complex classification problems (Chakravarthi et al., 2020; Hande et al., 2020; Biradar et al., 2021; Premjith et al., 2026). Building on these advancements, our work combines structured document processing with confidence-based prediction aggregation to improve writing style recovery in LLM-generated long-form text.

## 3 Task and Dataset

### 3.1 Task Description

The Prompt Recovery for LLM Shared Task at *DravidianLangTech@ACL 2026* is defined as a multi-class document-level classification problem. Each input document is a version rewritten by a large language model (LLM) to reflect a specific writing style. The objective is to assign the correct style label from nine predefined categories: *Authoritative*, *Formal*, *Humorous*, *Informal*, *Inspiring*, *Optimistic*, *Persuasive*, *Pessimistic*, and *Serious*. Each document corresponds to exactly one ground-truth label. System performance is evaluated primarily using macro-averaged F1 score to ensure equal weighting across classes, while overall accuracy is reported as an additional performance metric.

### 3.2 Dataset

The dataset is provided in an Excel file where the training, validation, and testing sets are already separated. Each data point contains an ID, the original text, and the rewritten text in a new style, provided in the column labeled "CHANGE STYLE." The actual style for the training and validation sets is also provided in the column labeled "STYLE."

The testing set contains 301 labeled data points, and the class distribution is slightly unbalanced in the testing set, as shown in Table 1 below.

We also notice that Serious is the most frequent class (15.61%) compared to Pessimistic, which is the least frequent (7.31%). We also observe that our test set is balanced across the nine classes, which

| Style | Support | % of Test |
|---|---|---|
| Authoritative | 30 | 9.97 |
| Formal | 38 | 12.62 |
| Humorous | 31 | 10.30 |
| Informal | 35 | 11.63 |
| Inspiring | 37 | 12.29 |
| Optimistic | 33 | 10.96 |
| Persuasive | 28 | 9.30 |
| Pessimistic | 22 | 7.31 |
| Serious | 47 | 15.61 |
| **Total** | **301** | **100** |

Table 1: Label distribution in the test set ($N = 301$).

is expected given that macro F1 is our main metric. We only trained our model on the "CHANGE STYLE" (style-rewritten) text column instead of the original text, as it was empirically shown that this column contains more explicit stylistic information. The rewriting of the text makes the signal for classification stronger.

## 4 System Design and Methodology

Figure 1 illustrates the end-to-end pipeline of the ByteBreaker system.

### 4.1 Model Architecture

We fine-tuned XLM-RoBERTa Large[1] from the Hugging Face Transformers library (Wolf et al., 2020) and used it as our base encoder. The Large variant is a 24-layer transformer with 1024-dimensional hidden states and roughly 560M parameters, pre-trained on a 2.5 TB multilingual corpus spanning nearly 100 languages. On top of the final `[CLS]` representation, we attach a sequence classification head—a linear layer that maps from 1024 to 9 logits, followed by a softmax. We opted for the Large model rather than the base version because its greater capacity helps capture subtle distinctions between closely related style categories.

### 4.2 Sliding-Window Chunking

Many documents in our dataset are longer than XLM-RoBERTa's 512-token input limit. Instead of truncating them and potentially losing important information, we use a sliding-window chunking method with the Hugging Face tokenizer's overflow handling. We tokenize each document and split it into overlapping chunks of up to 512 tokens, with a stride of 256 tokens (i.e., 50% overlap between consecutive chunks). Enabling `return_overflowing_tokens=True`, the tokenizer generates all chunks automatically and provides an `overflow_to_sample_mapping` array that records which original document each chunk comes from.

Each chunk inherits the label of its parent document. During training, we treat these chunks as independent labeled examples, so the model learns from every part of the original text. Using a 50% overlap also helps preserve context at chunk boundaries: tokens near the edges appear in two consecutive chunks, reducing the risk of losing important surrounding information.

### 4.3 Top-$K$ Mean Pooling

At inference time, a document that is split into $N$ chunks produces $N$ logit vectors, each in $\mathbb{R}^9$. To turn these chunk-level outputs into a single document-level prediction, we compare three aggregation strategies:

- **Mean Pooling:** averages all chunk logit vectors uniformly.
- **LogSumExp Pooling:** a soft maximum that gives more weight to high-confidence chunks.
- **Top-$K$ Mean Pooling (ours):** selects the $K$ chunks with the highest prediction confidence $\max_c L_{i;c}$ and averages only those $K$ logit vectors.

Formally, for a document $d$ with chunks $\{c_1, \ldots, c_N\}$ and corresponding logit vectors $\{L_1, \ldots, L_N\} \subset \mathbb{R}^9$, we define:

$$S_d = \text{top-}K \text{ chunks by } \max_c L_{i;c},$$
$$\hat{y}_d = \arg\max_j \frac{1}{|S_d|} \sum_{i \in S_d} L_{i;j}. \quad (1)$$

We set $K = \min(3, N)$ to gracefully handle short documents. In our validation experiments, Top-$K$ Mean Pooling with $K = 3$ consistently outperformed the other strategies, likely because it focuses on the most stylistically informative chunks and reduces the influence of ambiguous or mixed-style passages.

### 4.4 Training Configuration

We train our models using the Hugging Face `Trainer` API. Table 2 summarizes the key hyperparameters. For model selection, we monitor

[1] https://huggingface.co/FacebookAI/xlm-roberta-large

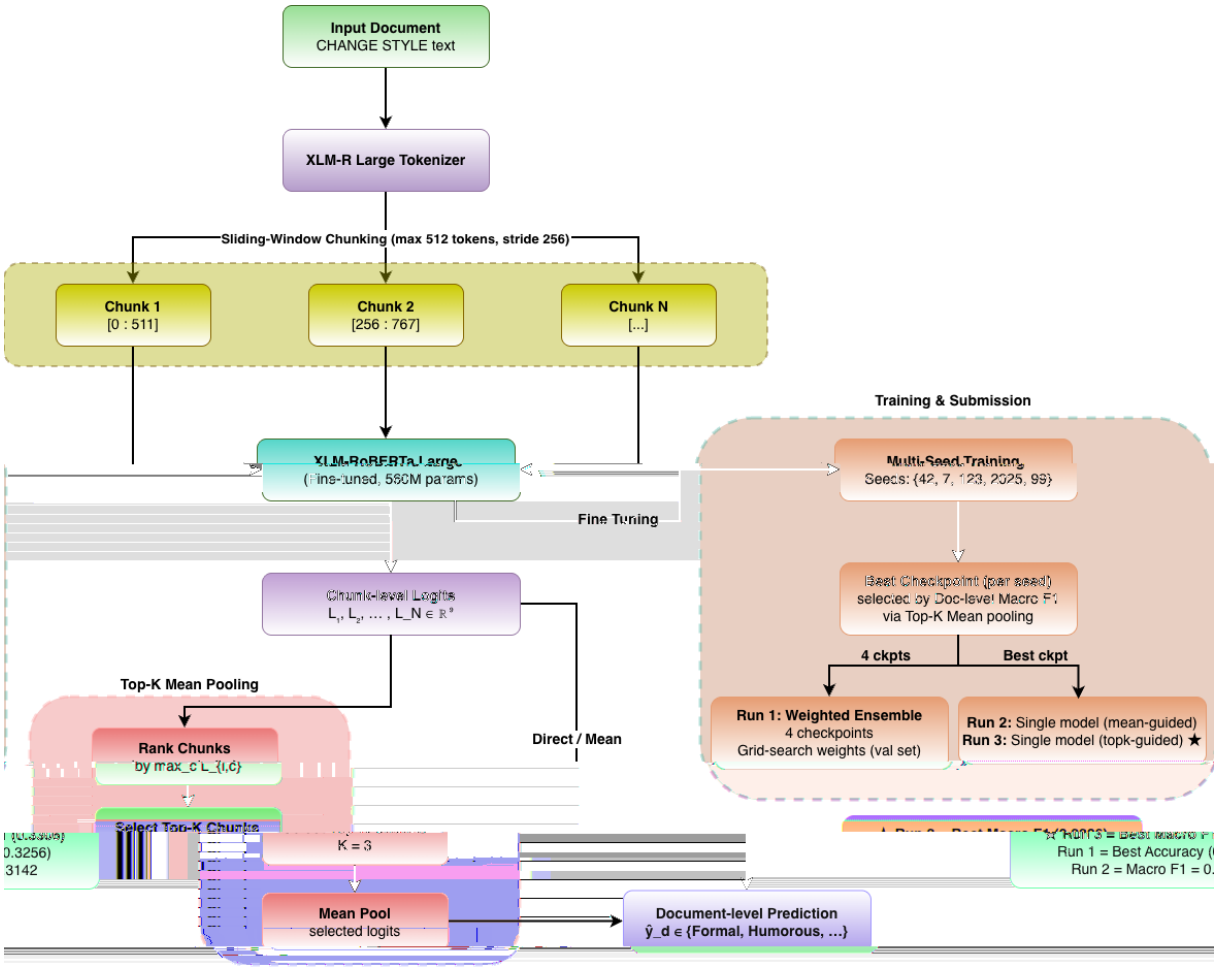


Figure 1: End-to-end pipeline of the ByteBreaker system. Documents exceeding 512 tokens are split into overlapping chunks (stride = 256). Each chunk is encoded independently by the fine-tuned XLM-RoBERTa Large. At inference, chunk-level logits are ranked by confidence and the top-$K$ are averaged into a document-level prediction. Five independently seeded models are trained; their best checkpoints feed into either a weighted ensemble (Run 1) or single-model submissions (Runs 2–3).

*document-level* macro F1—computed via Top-$K$ Mean pooling in a custom callback—rather than chunk-level accuracy, so the selection criterion directly matches the final evaluation metric. We apply label smoothing ($\varepsilon = 0.10$) to reduce overconfidence around noisy stylistic boundaries, and we use early stopping (patience $= 2$ epochs) to limit overfitting.

### 4.5 Multi-Seed Training

We train models across five random seeds: $\{42, 7, 123, 2025, 99\}$. For each seed, we export the checkpoint that achieves the best validation document-level macro F1. We then select the best checkpoint across all seeds as our primary model export. Using multiple seeds also gives us a diverse set of strong checkpoints, which we later leverage in our weighted ensemble.

### 4.6 Weighted Ensemble (Run 1)

For Run 1, we build a weighted ensemble of four XLM-RoBERTa Large checkpoints:

- **Model A:** Best checkpoint from the multi-seed training runs.
- **Model B:** Checkpoint from a separate single-seed run trained with Top-$K$ Mean evaluation guidance.
- **Model C:** `checkpoint-4062`—trained with Top-$K$ Mean pooling guidance (also used for **Run 3**).
- **Model D:** `checkpoint-2022`—trained with mean-pooling guidance (also used for **Run 2**).

Each model first produces document-level logit vectors using Top-$K$ Mean pooling. We then com-

| Hyperparameter | Value |
|---|---|
| Base model | xlm-roberta-large |
| Max sequence length | 512 tokens |
| Chunk stride | 256 tokens |
| Learning rate | $1 \times 10^{-5}$ |
| Effective batch size | 16 ($2 \times 8$ grad. accum.) |
| Max epochs | 12 |
| Warmup ratio | 0.10 |
| Weight decay | 0.01 |
| Gradient clipping | 1.0 |
| Label smoothing ($''$) | 0.10 |
| Best model metric | Doc-level macro F1 |
| Early stopping patience | 2 epochs |
| Mixed precision | BF16 |
| Seeds | {42, 7, 123, 2025, 99} |

Table 2: Training hyperparameters.

bine the models with a convex weighted sum:

$$L_{\text{ens}} = \sum_{m=1}^{4} w_m \cdot L_m, \quad \sum_{m=1}^{4} w_m = 1,\ w_m \geq 0. \tag{2}$$

We choose the weights $w$ using a two-stage grid search on the validation set: a coarse search with step size 0.10, followed by a finer search with step size 0.02 within a radius of 0.10 around the best coarse solution.

### 4.7 Submission Runs

- **Run 1:** Weighted ensemble of Models A–D with validation- optimized weights, using Top-$K$ Mean pooling for each model.
- **Run 2:** Single model—Model D (mean-guided)—with Top-$K$ Mean pooling at inference.
- **Run 3:** Single model—Model C (Top-$K$ guided)—with Top-$K$ Mean pooling at inference. *Best macro F1 overall.*

## 5 Results and Discussion

Tables 3 and 4 summarise official results on the 301-sample test set. Run 3 achieves the best macro F1 (**0.3306**) while Run 1 attains the highest accuracy (**0.3256**); the gap arises because accuracy favours the most frequent class (Serious, $N = 47$), whereas macro F1 weights all nine classes equally.

**Easiest classes.** Optimistic and Humorous achieve the highest F1 ($\approx$0.39–0.47) due to distinctive lexical markers.

**Hardest class.** Serious is consistently the weakest (F1 $\approx$ 0.10–0.14); it shares a measured formal register with Authoritative and Formal that the model cannot reliably separate.

| Run | Acc. | P | R | F1 |
|---|---|---|---|---|
| Run 1 (Ensemble) | **0.3256** | 0.3670 | 0.3400 | 0.3290 |
| Run 2 (mean-guided) | 0.3189 | 0.3572 | 0.3342 | 0.3142 |
| Run 3 (topk-guided)† | 0.3123 | 0.3524 | 0.3284 | **0.3306** |

Table 3: Official results (macro-averaged). † Best macro F1.

**Informal over-prediction.** Run 2 shows high Informal recall (0.66) but low precision (0.25); Run 3 achieves better balance ($P = 0.48$, $R = 0.31$).

**Why Run 3 outperforms the ensemble.** Including Model D in the ensemble dilutes Pessimistic precision from 0.50 to 0.30, reducing macro F1. A checkpoint aligned with the inference metric outperforms a heterogeneous ensemble when one member is weaker on minority classes.

## 6 Conclusion and Future Work

We introduced ByteBreaker for the Prompt Recovery task at *DravidianLangTech@ACL 2026*, which focuses on solving the problem of writing style classification of long documents by using structured document processing and prediction aggregation by confidence. Out of our runs, the best macro F1 score of 0.3306 is reported by Top-K Mean with single model (Run 3), and the best accuracy of 0.3256 is reported by the weighted ensemble (Run 1) with a macro F1 score of 0.3290.

As can be seen from the experimental results, the alignment between the training criterion and the inference strategy is crucial for the model's performance. Although the ensemble outperformed the single Top-K-guided model in terms of overall accuracy, the latter achieved a better macro F1 score by improving the performance on the minority classes. This implies that the confidence-aware chunk selection is more important than the uniform aggregation for the long document style prediction.

## 7 Ethics

This work uses data provided by the shared task organisers and publicly available pre-trained models. No personal data is collected or processed. Writing style classification carries potential risks of misuse in automated content profiling or manipulation; we encourage responsible deployment of such systems. We are committed to transparent reporting of methods and results, including per-class breakdowns that reveal model limitations. Our participation in the *DravidianLangTech@ACL-2026* shared task

| | | Run 1 | | | Run 2 | | | Run 3 | | |
|---|---|---|---|---|---|---|---|---|---|---|
| **Style** | **Supp.** | P | R | F1 | P | R | F1 | P | R | F1 |
| Authoritative | 30 | 0.33 | 0.20 | 0.25 | 0.28 | 0.17 | 0.21 | 0.18 | 0.23 | 0.20 |
| Formal | 38 | 0.28 | 0.24 | 0.26 | 0.32 | 0.24 | 0.27 | 0.28 | 0.26 | 0.27 |
| Humorous | 31 | 0.60 | 0.29 | 0.39 | 0.56 | 0.29 | 0.38 | 0.50 | 0.32 | 0.39 |
| Informal | 35 | 0.26 | 0.60 | 0.37 | 0.25 | 0.66 | 0.36 | 0.48 | 0.31 | 0.38 |
| Inspiring | 37 | 0.53 | 0.22 | 0.31 | 0.55 | 0.16 | 0.25 | 0.44 | 0.30 | 0.35 |
| Optimistic | 33 | 0.44 | 0.52 | 0.47 | 0.39 | 0.52 | 0.44 | 0.41 | 0.55 | 0.47 |
| Persuasive | 28 | 0.41 | 0.46 | 0.43 | 0.37 | 0.46 | 0.41 | 0.29 | 0.46 | 0.36 |
| Pessimistic | 22 | 0.30 | 0.41 | 0.35 | 0.33 | 0.41 | 0.37 | **0.50** | 0.41 | **0.45** |
| Serious | 47 | 0.15 | 0.13 | 0.14 | 0.17 | 0.11 | 0.13 | 0.10 | 0.11 | 0.10 |
| **Macro** | 301 | 0.37 | 0.34 | 0.33 | 0.36 | 0.33 | 0.31 | 0.35 | 0.33 | 0.33 |

Table 4: Per-class P, R, and F1 for all three runs.

has been conducted with ethical standards in mind, promoting collaboration and knowledge exchange within the research community.

## References


Shlomo Argamon, Casey Whitelaw, Paul Chase, Sobhan Raj Hota, Navendu Garg, and Shlomo Levitan. 2007. Stylistic text classification using functional lexical features. *Journal of the American Society for Information Science and Technology*, 58(6):802–822.

Iz Beltagy, Matthew E Peters, and Arman Cohan. 2020. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*.

Shankar Biradar, Sunil Saumya, and Arun Chauhan. 2021. mBERT based model for identification of offensive content in south indian languages. In *Working Notes of FIRE 2021 – Forum for Information Retrieval Evaluation*.

Bharathi Raja Chakravarthi, Vigneshwaran Muralidaran, Ruba Priyadharshini, and John Philip McCrae. 2020. Corpus creation for sentiment analysis in code-mixed tamil-english text. In *Proceedings of the 1st Joint Workshop on Spoken Language Technologies for Under-resourced languages (SLTU) and Collaboration and Computing for Under-Resourced Languages (CCURL)*, pages 202–210.

Alexis Conneau, Kartikay Khandelwal, Naman Goyal, Vishrav Chaudhary, Guillaume Wenzek, Francisco Guzmán, Édouard Grave, Myle Ott, Luke Zettlemoyer, and Veselin Stoyanov. 2020. Unsupervised cross-lingual representation learning at scale. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 8440–8451.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long and Short Papers)*, pages 4171–4186.

Shaik Fharook, Syed Sufyan Ahmed, Gurram Rithika, Sumith Sai Budde, Sunil Saumya, and Shankar Biradar. 2022. Are you a hero or a villain? A semantic role labelling approach for detecting harmful memes. In *Proceedings of the Workshop on Combating Online Hostile Posts in Regional Languages during Emergency Situations*, pages 19–23.

Adeep Hande, Ruba Priyadharshini, and Bharathi Raja Chakravarthi. 2020. KanCMD: Kannada codemixed dataset for sentiment analysis and offensive language detection. In *Proceedings of the Third Workshop on Computational Modeling of People's Opinions, Personality, and Emotion's in Social Media*, pages 54–63.

Sanjana M Kavatagi, Rashmi R Rachh, and Shankar S Biradar. 2023. VTUBGM@LT-EDI-2023: Hope speech identification using layered differential training of ULMFiT. In *Proceedings of the Third Workshop on Language Technology for Equality, Diversity and Inclusion*, pages 209–213.

Moshe Koppel, Jonathan Schler, and Shlomo Argamon. 2009. Computational methods in authorship attribution. *Journal of the American Society for Information Science and Technology*, 60(1):9–26.

Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. RoBERTa: A robustly optimized BERT pretraining approach. *arXiv preprint arXiv:1907.11692*.

B Premjith, Bharathi Raja, and Prasanna Kumar Kumaresan. 2026. Findings of the shared task on prompt recovery for LLM in Telugu – DravidianLangTech@ACL 2026. In *Proceedings of the Sixth Workshop on Speech and Language Technologies for Dravidian Languages (DravidianLangTech-2026)*, San Diego, California, USA. Association for Computational Linguistics. To appear.

Sunil Saumya, Vanshita Jha, and Shankar Biradar. 2022. Sentiment and homophobia detection on YouTube using ensemble machine learning techniques. In *Working Notes of FIRE 2022 – Forum for Information Retrieval Evaluation (Hybrid)*. CEUR.

Prasha Shrestha, Sebastian Sierra, Fabio Gonzalez, Manuel Montes, Paolo Rosso, and Thamar Solorio. 2017. Convolutional neural networks for authorship attribution of short texts. In *Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 669–674.

Chi Sun, Xipeng Qiu, Yige Xu, and Xuanjing Huang. 2019. How to fine-tune BERT for text classification? In *Proceedings of the 18th Chinese National Conference on Computational Linguistics (CCL)*, pages 194–206.

Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Remi Louf, Morgan Funtowicz, et al. 2020. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45.

Zichao Yang, Diyi Yang, Chris Dyer, Xiaodong He, Alexander Smola, and Eduard Hovy. 2016. Hierarchical attention networks for document classification. In *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 1480–1489.

Manzil Zaheer, Guru Guruganesh, Kumar Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, et al. 2020. Big Bird: Transformers for longer sequences. *Advances in Neural Information Processing Systems*, 33:17283–17297.